

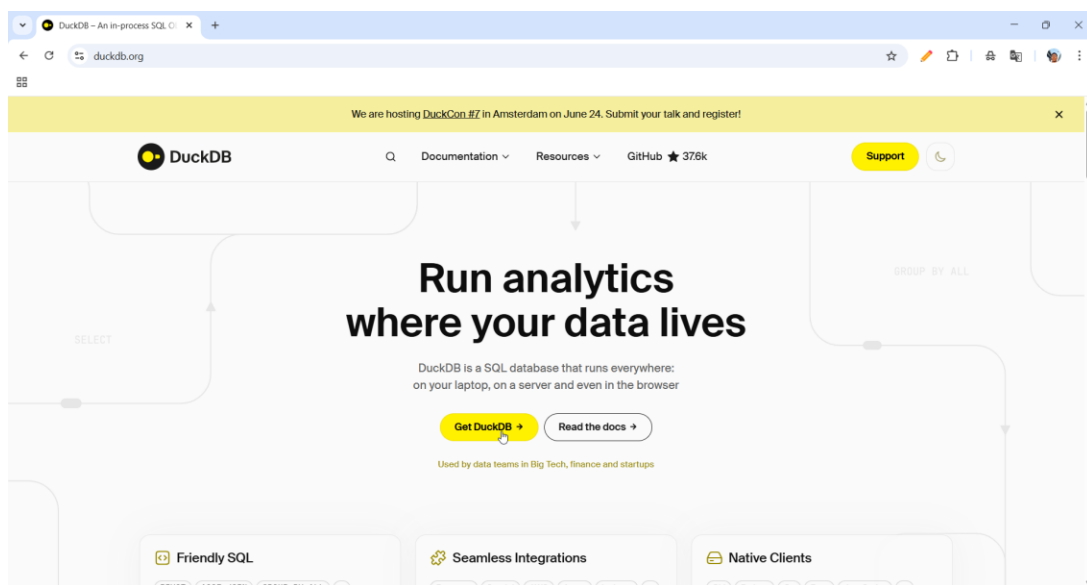
Data Engineer Assignment – MIFX

Alexander Prasetyo Christianto

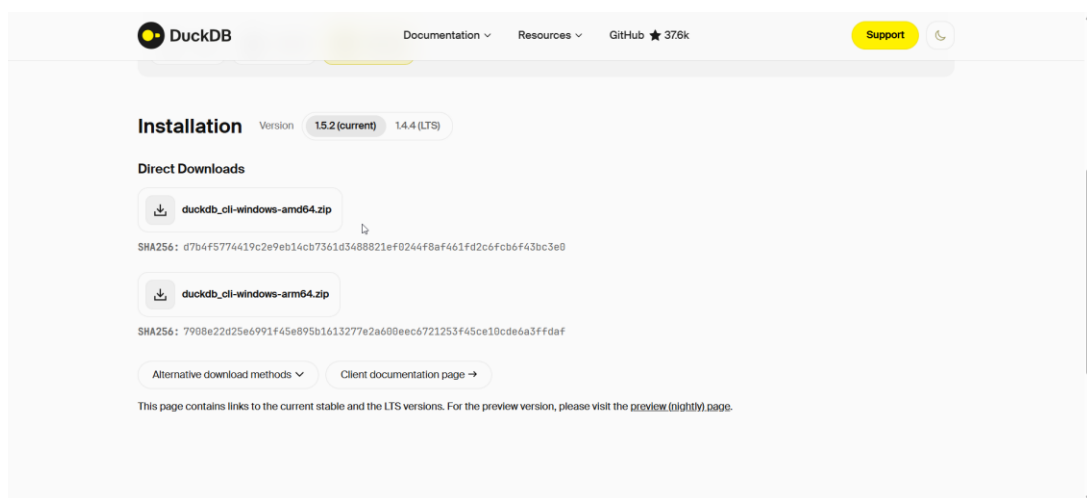
Case 1

After reading and analyzing the first case, I decided to proceed using DuckDB to help me visualize and get the full use of the data. To enhance the readability of this document, I provide the steps and processes I went through.

1. Go to the official website.



2. Download and install the client.



3. Extract the downloaded zip and execute .exe file.


```
memory D
memory D CREATE TABLE "DATA_PAYMENT" AS SELECT * FROM read_csv('pay.csv', delim=';', header=true, nullstr='');
memory D CREATE TABLE "DATA_USER" AS SELECT * FROM read_csv('user.csv', delim=';', header=true, nullstr='');
memory D select * from data_payment limit 5;
```

payment_id int64	order_id int64	pay_method varchar	pay_status varchar	gmt_create varchar	gmt_modified varchar
200166000502091961	100166000002948315	BALANCE	SUCCESS	28/09/2025 20:34	28/09/2025 20:34
2001660009002241346	100166000003103124	BALANCE	SUCCESS	13/10/2025 20:05	13/10/2025 20:05
2001660006302082194	100166000102942144	BALANCE	SUCCESS	28/09/2025 15:48	28/09/2025 15:48
2001660003102256854	100166000103112464	BALANCE	SUCCESS	14/10/2025 19:55	14/10/2025 19:55
2001660009502076255	100166000202935695	BALANCE	SUCCESS	27/09/2025 18:40	27/09/2025 18:40

```
memory D select * from data_user limit 5;
```

user_id varchar	registration_site varchar	registration_platform varchar	is_premium_user boolean	registration_time varchar
4270000007072	Mobile	plat_3	false	26/07/2025 15:21
4270000007072	Mobile	plat_3	false	26/07/2025 15:21
4270000007072	Mobile	plat_3	false	26/07/2025 15:21
4270000007072	Mobile	plat_3	false	26/07/2025 15:21
4270000007072	Mobile	plat_3	false	26/07/2025 15:21

```
memory D
```

NOTE: I added "DATA_" for the name of the tables because order is a keyword for the SQL.

Before moving on to create the staging table, I decided to do a simple check on the data related to the given instruction for this case. Below is the result of `.tables`` function of DuckDB for the created tables;

```
mifxcase D .tables
```

data_order		data_payment		data_user	
order_id	varchar	payment_id	varchar	user_id	varchar
symbol_id	varchar	order_id	varchar	registration_site	varchar
buyer_id	varchar	pay_method	varchar	registration_platform	varchar
order_status	varchar	pay_status	varchar	is_premium_user	varchar
order_platform	varchar	gmt_create	varchar	registration_time	varchar
gmt_create	varchar	gmt_modified	varchar		
gmt_modified	varchar				
720465 rows		71983 rows		588217 rows	

And with the help of using python, I checked the sum of null values for each table.

```
df_order.isnull().sum()
```

[10] ✓ 0.1s

```
... order_id      0
    symbol_id    69
    buyer_id     0
    order_status  0
    order_platform 85
    gmt_create    0
    gmt_modified  0
    dtype: int64
```

```
df_payment.isnull().sum()
```

[13] ✓ 0.0s

```
... payment_id    0
    order_id      0
    pay_method    158
    pay_status    0
    gmt_create    0
    gmt_modified  0
    dtype: int64
```

▶ ▾

```
df_user.isnull().sum()
```

[14] ✓ 0.0s

```
... user_id      0
    registration_site 0
    registration_platform 219
    is_premium_user  0
    registration_time  0
    dtype: int64
```

There is a difference between using python and sql from counting the sum of null values. Here is the result of counting the null values from each table in DuckDB

```
mifxcase D -- Null counts for data_payment
mifxcase D SELECT
    COUNT(*) FILTER (WHERE pay_method IS NULL) AS pay_method_nulls
FROM data_payment;
```

pay_method_nulls
0

```
mifxcase D
mifxcase D -- Null counts for data_user
mifxcase D SELECT
    COUNT(*) FILTER (WHERE registration_platform IS NULL) AS registration_platform_nulls
FROM data_user;
```

registration_platform_nulls
0

```
mifxcase D
mifxcase D
mifxcase D select distinct pay_method from data_payment;
```

pay_method
COUPON
DIRECT_DEBIT_DEBIT_CARD
DEBIT_CARD
CREDIT_CARD
BALANCE
VIRTUAL_ACCOUNT
null

To prevent this from creating bigger problems in the future, I will drop table of data_user and data_payment and recreate the table and treating 'null' as a null value by using this query

```
CREATE TABLE data_payment ( payment_id VARCHAR, order_id VARCHAR, pay_method
VARCHAR, pay_status VARCHAR, gmt_create VARCHAR, gmt_modified VARCHAR ); COPY
data_payment FROM 'pay.csv' (DELIMITER ';;' HEADER true, NULLSTR 'null');
```

For table data_user, there is a little bit of difference because NULL values are store in a form of string = 'NULL'. While, in data_payment NULL values are = 'null'. So I updated the table by using this:

```
CREATE TABLE data_user ( user_id VARCHAR, registration_site VARCHAR,
registration_platform VARCHAR, is_premium_user VARCHAR, registration_time VARCHAR
); COPY data_user FROM 'user.csv' (DELIMITER ';;' HEADER true);
```

```
-- Convert both 'null' and 'NULL' string variants to proper NULL
```

UPDATE data_user SET registration_platform = NULL WHERE LOWER(registration_platform) = 'null';

Here is the updated result and it is now matched the python result

```
mifxcase D -- Null counts for data_order
mifxcase D SELECT
    COUNT(*) FILTER (WHERE symbol_id IS NULL) AS symbol_id_nulls,
    COUNT(*) FILTER (WHERE order_platform IS NULL) AS order_platform_nulls
FROM data_order;
```

symbol_id_nulls int64	order_platform_nulls int64
69	85

```
mifxcase D
mifxcase D -- Null counts for data_payment
mifxcase D SELECT
    COUNT(*) FILTER (WHERE pay_method IS NULL) AS pay_method_nulls
FROM data_payment;
```

pay_method_nulls int64
158

```
mifxcase D
mifxcase D -- Null counts for data_user
mifxcase D SELECT
    COUNT(*) FILTER (WHERE registration_platform IS NULL) AS registration_platform_nulls
FROM data_user;
```

registration_platform_nulls int64
219

The next step is creating the table summary_trx_user_count

CREATE TABLE IF NOT EXISTS summary_trx_user_count (report_date DATE,
dimension_type VARCHAR(50), dimension_value VARCHAR(100), all_trx_count BIGINT,
all_trx_user_count BIGINT, success_trx_count BIGINT, success_trx_user_count BIGINT);

The next step is to inject the data into the staging table. Below is the final query:

```
INSERT INTO summary_trx_user_count
WITH
/* Step 1: Create a date spine to ensure all dates are represented.
This also ensures that all dates with no orders in the requested
time window keep appearing in the table. */
date_spine AS (
    SELECT UNNEST(
        generate_series('2025-09-01'::timestamp, '2025-10-31'::timestamp, INTERVAL '1 day')
    )::date AS report_date
),
/* Step 2: Create a base CTE that combines order, payment, and user data.
Things to note here:
1. Join logic:
```

- data_order LEFT JOIN data_payment -> keeps orders with no payment record
- data_order LEFT JOIN data_user -> keeps orders whose buyer has no user record

2. NULL value handling:

- NULL values in the joined tables will be replaced with 'OTHERS' as stated in the instructions.
- This applies to: order_platform, symbol_id, pay_method, and is_premium_user.
- NULLs can come from two sources:
 - a. The column itself is NULL in the source table (already cleaned during table creation)
 - b. No matching row found in the LEFT JOINed table (buyer has no user record, order has no payment)

3. Date filter:

- Only orders within the reporting window (2025-09-01 to 2025-10-31) are included.
- gmt_create is stored as VARCHAR, so we parse it using strptime() before filtering.

```

*/
base AS (
  SELECT
    o.order_id,
    o.buyer_id,
    o.order_status,
    strptime(o.gmt_create, '%d/%m/%Y %H:%M')::date      AS report_date,
    COALESCE(o.order_platform, 'OTHERS')                AS order_platform,
    COALESCE(o.symbol_id, 'OTHERS')                    AS symbol_id,
    COALESCE(p.pay_method, 'OTHERS')                   AS pay_method,
    COALESCE(u.is_premium_user, 'OTHERS')               AS is_premium_user
  FROM data_order o
  LEFT JOIN data_payment p ON o.order_id = p.order_id
  LEFT JOIN data_user u   ON o.buyer_id = u.user_id
  WHERE strptime(o.gmt_create, '%d/%m/%Y %H:%M')::date
        BETWEEN '2025-09-01' AND '2025-10-31'
),

```

```

/* Step 3: Aggregate base CTE per dimension type.
Each CTE below slices the base data by one dimension and computes 4 metrics:
- all_trx_count      : total number of orders regardless of status
- all_trx_user_count : number of unique buyers regardless of status
- success_trx_count  : number of orders with status = 'SUCCESS'
- success_trx_user_count : number of unique buyers with status = 'SUCCESS'

```

Note: dim_order_platform has explicit column aliases to serve as the column name reference for all subsequent UNION ALL CTEs. The rest omit aliases since UNION ALL inherits column names from the first SELECT. */

```

-- Slice by order_platform
dim_order_platform AS (
  SELECT
    report_date,
    'order_platform' AS dimension_type,
    order_platform AS dimension_value,
    COUNT(order_id) AS all_trx_count,
    COUNT(DISTINCT buyer_id) AS all_trx_user_count,
    COUNT(CASE WHEN order_status = 'SUCCESS' THEN order_id END) AS success_trx_count,

```

```

        COUNT(DISTINCT CASE WHEN order_status = 'SUCCESS' THEN buyer_id END) AS
success_trx_user_count
    FROM base
    GROUP BY report_date, order_platform
),

-- Slice by symbol_id
dim_symbol_id AS (
    SELECT
        report_date,
        'symbol_id' AS dimension_type,
        symbol_id AS dimension_value,
        COUNT(order_id),
        COUNT(DISTINCT buyer_id),
        COUNT(CASE WHEN order_status = 'SUCCESS' THEN order_id END),
        COUNT(DISTINCT CASE WHEN order_status = 'SUCCESS' THEN buyer_id END)
    FROM base
    GROUP BY report_date, symbol_id
),

-- Slice by pay_method
dim_pay_method AS (
    SELECT
        report_date,
        'pay_method' AS dimension_type,
        pay_method AS dimension_value,
        COUNT(order_id),
        COUNT(DISTINCT buyer_id),
        COUNT(CASE WHEN order_status = 'SUCCESS' THEN order_id END),
        COUNT(DISTINCT CASE WHEN order_status = 'SUCCESS' THEN buyer_id END)
    FROM base
    GROUP BY report_date, pay_method
),

-- Slice by is_premium_user
dim_is_premium_user AS (
    SELECT
        report_date,
        'is_premium_user' AS dimension_type,
        is_premium_user AS dimension_value,
        COUNT(order_id),
        COUNT(DISTINCT buyer_id),
        COUNT(CASE WHEN order_status = 'SUCCESS' THEN order_id END),
        COUNT(DISTINCT CASE WHEN order_status = 'SUCCESS' THEN buyer_id END)
    FROM base
    GROUP BY report_date, is_premium_user
),

/* Step 4: Stack all 4 dimension aggregations into one result set using UNION ALL.
UNION ALL is used instead of UNION because we want to keep all rows
(no deduplication needed since each CTE produces different dimension_type values). */
all_dims AS (
    SELECT * FROM dim_order_platform

```



```

        UNION ALL SELECT * FROM dim_symbol_id
        UNION ALL SELECT * FROM dim_pay_method
        UNION ALL SELECT * FROM dim_is_premium_user
    )

/* Step 5: Final SELECT - LEFT JOIN date_spine with all_dims.
   - date_spine drives the output, guaranteeing every date appears even with zero
   transactions.
   - COALESCE on metric columns converts NULL to 0 for dates with no transactions.
   - Results are ordered by date, then dimension type, then dimension value for readability.
*/
SELECT
    ds.report_date,
    d.dimension_type,
    d.dimension_value,
    COALESCE(d.all_trx_count, 0) AS all_trx_count,
    COALESCE(d.all_trx_user_count, 0) AS all_trx_user_count,
    COALESCE(d.success_trx_count, 0) AS success_trx_count,
    COALESCE(d.success_trx_user_count, 0) AS success_trx_user_count
FROM date_spine ds
LEFT JOIN all_dims d ON ds.report_date = d.report_date
ORDER BY ds.report_date, d.dimension_type, d.dimension_value;

```